

FINETUNING SMALL LANGUAGE MODELS FOR JAVANESE TRANSLATION

TRIVAN THAMRIN MENEZES

ADVISOR: PROFESSOR CHRISTIANE FELLBAUM

SUBMITTED IN PARTIAL FULFILLMENT
OF THE REQUIREMENTS FOR THE DEGREE OF
BACHELOR OF ARTS
DEPARTMENT OF COMPUTER SCIENCE
PRINCETON UNIVERSITY

APRIL 2025

I hereby declare that I am the sole author of this thesis.

I authorize Princeton University to lend this thesis to other institutions or individuals for the purpose of scholarly research.

Trivan Thamrin Menezes

I further authorize Princeton University to reproduce this thesis by photocopying or by other means, in total or in part, at the request of other institutions or individuals for the purpose of scholarly research.

Trivan Thamrin Menezes

Abstract

Natural language processing tools remain scarce for low-resource languages, despite their large speaker populations. This research investigates the potential for improving machine translation for Javanese, an Indonesian language with over 80 million speakers but limited digital resources. We analyze the performance of fine-tuning techniques—supervised fine-tuning (SFT), model distillation, and Chain-of-Thought (CoT) distillation—on enhancing Javanese-to-Indonesian translation quality using open-weight models (T5, mT5, Gemma 3 4B, Aya 8B, Aya 32B), comparing them against zero-shot and many-shot baselines from larger proprietary models.

Evaluation using BLEU, TER, chrF, and BERTScore reveals that while large models like Gemini 2.0 Flash achieve top performance, fine-tuning significantly boosts the performance of smaller models. Initial SFT on the 500-example NusaX dataset provided substantial gains. Notably, the fine-tuned 4-billion parameter Gemma 3 4B model achieved performance comparable to, and sometimes exceeding, much larger zero-shot models like GPT-4o. Subsequent model distillation and CoT distillation yielded only marginal improvements over SFT, suggesting diminishing returns potentially limited by pre-training knowledge. A key challenge identified was a formality mismatch in the training data.

The results demonstrate that fine-tuning smaller, accessible models offers a resource-efficient pathway to high-quality translation for low-resource languages like Javanese, potentially enabling deployment on edge devices and broadening access to NLP technologies for underserved linguistic communities.

Acknowledgements

...

(fill this in)

Contents

Abstract	iii
Acknowledgements	i
List of Tables	vi
List of Figures	viii
1 Introduction	1
1.1 Contributions	1
2 Related Work	3
2.1 Low-Resource NLP and Translation Projects	3
2.2 Translation Projects for Indonesian Languages	4
2.3 Machine Translation Models	4
2.3.1 Seq2Seq	4
2.3.2 Generative Models	5
2.3.3 Adapting Generative Models for Tasks	6
2.4 LLM Context Windows	8
2.4.1 Language Learning from Long Context	8
2.5 Fine-tuning Techniques	9
2.5.1 Supervised Fine-tuning	9
2.5.2 Model Distillation	11
2.5.3 Chain-of-Thought Distillation	11

3	Methods	13
3.1	Goal	13
3.2	Dataset	14
3.3	Models	15
3.3.1	Closed-Source Models	15
3.3.2	Open-Weight Models	16
3.4	Dataset Construction	16
3.4.1	Stage 1: Supervised Fine-Tuning on Initial Dataset	16
3.4.2	Stage 2: Model Distillation	17
3.4.3	Stage 3: Chain-of-Thought Distillation	19
3.5	Fine-Tuning	21
3.5.1	Memory Considerations	22
3.5.2	Parameter-Efficient Fine-tuning	24
3.5.3	Training Decisions	24
4	Evaluation	26
4.1	Evaluation Techniques	26
4.1.1	BLEU	27
4.1.2	TER	29
4.1.3	chrF	31
4.1.4	BERTScore	33
5	Discussion	38
5.1	Overall Performance	38
5.2	Comparison of Techniques	39
5.2.1	Takeaways	41
5.3	Common Issue: Formality Mismatch	42
5.4	Model Size Considerations	43

6	Conclusions and Future Work	45
6.1	Implications and Significance	46
6.2	Future Work	47
A	Code	48

List of Tables

3.1	Model Specifications. <i>Open</i> means that the model’s weights are available online publicly, and its licensing permits further training. <i>Single-GPU</i> indicates the model can be trained on a single GPU with sufficient memory. <i>Local</i> signifies that the model is small enough to be run locally on a laptop computer.	15
3.2	Summary of Constructed Datasets	21
4.1	BLEU scores - Model performance across different training configurations on a test set of 400 entries. Models are sorted by zero-shot performance.	29
4.2	TER scores - Model performance across different training configurations on a test set of 400 entries. Lower scores indicate better performance. Models are sorted by zero-shot performance.	31
4.3	chrF scores - Model performance across different training configurations on a test set of 400 entries. Higher scores indicate better performance. Models are sorted by zero-shot performance.	33
4.4	Median BERTScore values - Model performance across different training configurations using XLM-RoBERTa-base embeddings. Higher scores indicate better semantic similarity to reference translations. Models are sorted by zero-shot performance.	35

5.1	Example of pronoun formality differences in translation	43
-----	---	----

List of Figures

3.1	Training and validation loss curves for Gemma 3 4B model with Chain-of-Thought distillation. The checkpoint with the lowest validation loss was selected for evaluation.	25
4.1	BERTScore results for different models using xlm-roberta-base embeddings. The figure shows that Gemini 2.0 Flash with many-shot learning achieved the highest average, median, and minimum scores, while most models reached a maximum score of 1.0, indicating perfect matches with reference translations in some cases.	37

Chapter 1

Introduction

Most work in natural language processing (NLP) has catered to high-resource languages, which are languages for which there exists numerous data and datasets. There is a scarcity of NLP tools for low-resource languages, which are languages that do not have much data even if they may have a high number of speakers.

With over 700 languages used across the nation’s cultures, Indonesia is one of the most linguistically diverse regions of the world, yet most NLP work for Indonesian languages centers around Bahasa Indonesia, the official language of the country [31]. Bahasa Indonesia was created a century ago to unify the cultures of Indonesia, but with this language’s widespread adoption, numerous other Indonesian languages have declined in use and popularity [31]. That said, Indonesian languages such as Javanese still have around 84 million speakers[31]. Improving NLP tooling for Javanese may empower its community with greater access to global information, helping preserve the language while connecting its speakers to resources in other languages.

1.1 Contributions

The goal of this work is to improve translation models for these low-resource Indonesian languages, using techniques such as *supervised fine-tuning* (SFT) and *model*

distillation, which have been shown to improve the performance of small language models (SLMs) on various tasks [10] [16].

Improving the performance of translation models for Javanese while balancing resource use and model size may inspire similar work to improve the performance and efficiency of models for other low-resource languages across various NLP tasks.

Chapter 2

Related Work

2.1 Low-Resource NLP and Translation Projects

There is existing work on extending modern transformer-based language models (LMs) to cater to low-resource languages, but there **there are only a few ... ?** **are minimal** projects which focus on low-resource Indonesian languages directly.

Often, these projects have focused on pretraining LMs in an unsupervised manner on corpuses of unstructured data across many languages. Centralized efforts such as Cohere4AI's Aya model project have yielded progress for performance across broad categories of NLP tasks [7]. Meta's *No Language Left Behind* (NLLB) project also focused on machine translation for low-resourced languages, covering a few Indonesian languages (Acehnese, Javanese, Balinese, Banjar, Buginese, and **would you call this low-resource?** **Bahasa Indonesia**) [28]. NLLB showed that increasing the languages supported by a model and increasing its translation quality are not mutually exclusive goals. The work culminated in *NLLB-200*, a single AI model with the ability to translate 200 languages, as well as *FLORES-200*, a dataset to evaluate translation across 200 languages.

2.2 Translation Projects for Indonesian Languages

Centralized efforts have recently emerged to further NLP research around Indonesian languages. NusaCrowd¹ is a project to gather Indonesian language datasets in a unified repository, and NusaX is a multilingual parallel dataset for ten Indonesian languages [4] [31].

We utilize NusaX to expand language model translation capability through several fine-tuning approaches.

2.3 Machine Translation Models

Machine translation (MT) has evolved significantly in the past few decades, with evolution from statistical approaches to neural machine translation, and currently, generative models.

2.3.1 Seq2Seq

Sutskever et al. (2014) introduced sequence-to-sequence (Seq2Seq) models, which became the dominant implementation for MT models before large pre-trained transformers rose to dominance [24]. Seq2Seq models usually include an encoder and decoder layer. The encoder processes the input sequence (in source language) into a vector of a fixed-dimension. This vector is then passed into the decoder layer, which produces the output sequence in the target language.

Recurrent Neural Networks (RNNs) or Long Short-Term Memory (LSTM) network architectures were the initial primary implementations of Seq2Seq models, which process input sequences of various lengths [24] [5]. However, these sequential architectures had a limited context window size, as they compressed all previous information into fixed-size hidden states, causing information loss for longer sequences.

¹<https://github.com/IndoNLP/nusa-crowd>

This limitation was largely alleviated by the creation of the attention mechanism [2], which enabled the decoder to focus on relevant parts of the input sequence at each step and maintain access to the entire input context regardless of sequence length. The Transformer architecture [29], based solely on attention mechanisms, was no longer bottlenecked by as storing context into a small number of hidden states, and it also enabled parallel processing and direct connections between any positions in a sequence [2]. This greatly enhanced the performance of Seq2Seq models on translation and a variety of other natural language processing (NLP) tasks, becoming the foundation for many modern MT systems and large language models [29].

In this study, the Seq2Seq models we analyze are Google’s T5 and multilingual T5 (mT5) models. We compare the performance of these models on Javanese-to-Indonesian translation with generative language models.

2.3.2 Generative Models

More recently, large-scale generative language models, primarily based on the decoder-only Transformer architecture, such as the Generative Pre-trained Transformer (GPT series) [3, 21], show high performance across a variety of NLP tasks, including machine translation, often without explicit task-specific training (zero-shot) or with only a few examples (few-shot) [3] provided alongside the input text.

While not always explicitly trained as Seq2Seq models for translation, generative language models are extensively pre-trained on vast amounts of unstructured text data, enabling them to understand and generate realistic text in multiple languages [21]. These models often perform translation effectively when prompted appropriately [3]. That said, their performance generally caters toward high-resource languages, which often dominate the pre-training datasets, and may perform less effectively for low-resource languages that are not as well represented in the training data [26, 28].

2.3.3 Adapting Generative Models for Tasks

The general-purpose nature of generative language models contrasts with Seq2Seq models trained specifically for tasks, although multiple techniques exist for adapting them for higher performance on specific translation tasks.

There are two commonly used approaches for adapting general-purpose language models for task-specific use cases: in-context learning and fine-tuning.

In our research, we evaluate a set of generative language models, testing both in-context learning and fine-tuning approaches for the translation task and comparing their performance.

In-Context Learning

For the modern generation of generative pre-trained transformer (GPT) models, Brown et al. (2020) showed early on that these models can effectively learn new tasks from just a few examples provided in the input context, using GPT-3 as the evaluated model [3]. This work showed that instead of having to train individual task-specific models for a variety of tasks, one can use a generalized pre-trained model with adapted prompts and example sets to tailor the model to a particular use case with high performance. Larger models have increasingly become capable of inferring patterns from examples from prompts without requiring parameter updates [3].

The process of providing examples **in ?** into the input prompt for a language model at inference time has been given the name *in-context learning*, as the model “learns” how to perform the task from the input context without updating its parameters through extended training [26]. This approach is also commonly referred to as *few-shot* or *many-shot* prompting, depending on the number of examples provided. The primary drawback of this approach is the increased input prompt length, which significantly increases computational resources required and overall inference costs, especially when **a large number of examples are provided** providing a high number of examples.

This capability is relevant for translation tasks involving low-resource languages, where providing a few translation examples in the prompt can enable the model to generalize to new sentences. The model can adapt its behavior based on the patterns observed in these examples, allowing for flexible task adaptation without the computational cost of fine-tuning or extended training [26].

Fine-tuning

Meanwhile, *fine-tuning* involves updating the parameters of a pre-trained language model on a supervised (labeled) dataset to adapt it for a particular task. This process is essentially a continuation of the training process, but it is usually focused on task-specific use cases and output formatting conditions [21]. Fine-tuning typically requires less data and computational resources than training a model from scratch, as the model already possesses general language understanding capabilities [21, 14].

For translation tasks, fine-tuning allows models to learn the specific patterns and conventions of translating between particular language pairs. Fine-tuning can be performed on either the entire model or a subset of the weights through parameter-efficient methods that save compute resources without sacrificing high levels of performance [34].

The success of pre-training followed by fine-tuning was popularized by works such as ULMFiT [11] (though initially using LSTMs), **laid** the groundwork for the effectiveness of these large generative models. **something missing? "which" ?**

In this study, we test in-context learning and fine-tuning on various models for the task of **Javanese-Indonesian translation** and compare trade-offs between performance and model size.

Somewhere it would be useful to say more about the relationship between these two languages. Similar? Very different? That would make a difference in how well LLMs work.

2.4 LLM Context Windows

Since the introduction of large language models, their context windows have increased in size substantially. Context windows are the maximum number of tokens a model can process at once, where a **token is usually a word, phrase, or punctuation mark** **aren't they more often parts of words?** that acts as the basic unit of text processing for the model.

BERT (2018) [9] began with just 512 tokens, while GPT-2 (2019) [22] increased to 1,024 and GPT-3 (2020)[3] to 2,048. The context windows of recent models have grown exponentially: GPT-4 initially supported 8,192 tokens before expanding to 32,768 [17], GPT-4 Turbo supported 128,000, and Google’s recent Gemini models handle very large contexts, over a million tokens [7]. The expanded size of context windows enables models to process longer documents, maintain extended conversations in chat use cases, and incorporate more examples for in-context learning, which can be valuable for translation tasks requiring multiple examples.

2.4.1 Language Learning from Long Context

Recent research has explored the ability of large language models to learn new languages from information provided directly in the input context window. The *Machine Translation from One Book* (MTOB) benchmark [25] investigates whether LLMs can learn to translate between English and Kalamang using only several hundred pages of field linguistics reference materials. Kalamang is also a low-resource language from Indonesia, but it has fewer than 200 speakers and virtually no web presence [25]. This approach differs from traditional neural machine translation training, which usually involves training a model solely on large parallel corpora. Instead, MTOB frames language learning in a similar manner to human second language (L2) acquisition, where the model must learn grammar rules and vocabulary from explanations instead of only from massive examples [25] [26].

Upon the release of the Gemini 1.5 series of long-context language models, Google ran the MTOB benchmark by loading the Kalamang language guide into the context window. When loading half of the guide and the entire guide into context, Gemini 1.5 Flash and Pro received scores of 5.38 and 5.46 out of 6, respectively, which is almost equal to the quality of a human language learner, who scored 5.60 out of 6 [26].

In this study, we attempt a similar approach, but instead of loading a grammar guide into the context window, we use a **parallel corpus of a few hundred examples**. **from where?** In the Gemini MTOB evaluation, Google also utilized approximately 400 parallel sentences in context alongside the grammar guide [26], so we explore whether parallel examples alone are sufficient to improve low-resource language translation quality.

2.5 Fine-tuning Techniques

Fine-tuning is an effective approach for adapting pre-trained language models to specific tasks by training them on task-specific data. For machine translation, fine-tuning involves using parallel corpus data between specific language pairs. This study explores three fine-tuning approaches: Supervised Fine-tuning (SFT), Model Distillation, and Chain-of-Thought Distillation. We evaluate these techniques to assess their performance for Javanese-Indonesian language translation and their training efficiency.

2.5.1 Supervised Fine-tuning

Supervised Fine-tuning (SFT) is a continuation of the training process that adapts pre-trained language models to specific tasks using labeled data. For translation, this involves training on parallel text in source and target languages [34].

SFT requires fewer resources than training from scratch [21], making it practical for specialized applications. In our experiments, we apply SFT to various pre-trained

models using our Javanese-Indonesian corpus to evaluate both their adaptability and the limitations imposed by their pre-training knowledge.

Pre-training Limitations and Data Efficiency

While SFT can instill new knowledge into models, there may exist a performance ceiling determined by the model’s pre-training phase. Models with limited exposure **limited? little? not much?** to certain languages during pre-training often show **constrained** improvement during fine-tuning on those languages [28]. This is particularly relevant for low-resource languages like Javanese, where the model’s initial understanding may be limited.

Research by Zhu et al. (2024) on fine-tuning data requirements revealed that even with just 32 examples, LLMs can achieve substantial translation improvements for well-represented languages [34]. Their experiments showed the most significant gains occur in early scaling (8 to 32 examples), with diminishing returns beyond 128 examples.

This supports the “superficial alignment hypothesis” (Zhou et al., 2023), which claims that models acquire knowledge primarily during pre-training, while fine-tuning **what is that in the translation context?** mainly teaches **response formatting** rather than new knowledge [33].

In our study, we conduct experiments with SFT using datasets of varying sizes and compositions. We begin with a small base dataset D , then evaluate performance when D is augmented with synthetic data generated by a large language model (Model Distillation), and finally when D is enhanced with explanatory content (Chain-of-Thought Distillation). The following sections provide detailed descriptions of these approaches and their implementation.

Through these experiments, we aim to investigate the assertions about the importance of data quantity and quality in the fine-tuning process for low-resource language translation tasks.

2.5.2 Model Distillation

Model distillation (also known as *knowledge distillation*) is a type of SFT, aiming to capture the performance of a large language model within a smaller model.

The technique involves using the **sounds incestuous to me** output of a *large* language model (the “teacher”) to train a *small* language model (the “student”) [16, 14]. Since the training dataset is generated by another model, the dataset is considered to be synthetic, as its creation did not utilize manual data labeling by a human.

In this process, the teacher model creates a probability distribution from the initial dataset through its training. The teacher model generates idealized examples that help the student model capture this distribution effectively. The large teacher model can achieve a “deeper understanding” of the task due to its substantial parameter count, but many of these parameters may be unnecessary when the model’s use is scoped to a particular task like translation, i.e. a less generalized, small model could be able to approximate a similar level of performance. This makes using a smaller model sensible, and model distillation (training on the large model’s outputs) is the method to achieve high performance with a more compact model.

Other studies demonstrate that this approach improves performance on various tasks, such as mathematical reasoning [14]. A common implementation strategy is to augment an existing dataset by adding more examples generated by the teacher model [14].

However, Zhu et al. (2024) argues that training models on synthetic data produced by an LLM may lead to **my observation above** overfitting on the imperfections of the noisy synthetic data in some cases, producing suboptimal translation quality.

2.5.3 Chain-of-Thought Distillation

Recent studies indicate that including explicit reasoning steps into training data can improve model training efficiency [12, 14]. By including a step-by-step reasoning

sequence, or a *chain-of-thought* [30], into the training data during distillation, these studies have shown that language models can achieve comparable performance on certain tasks with substantially fewer training examples compared to traditional training on a greater number of examples [12, 14].

Wei et al. (2023) [30] first introduced *chain-of-thought* prompting, which gives rise to more sophisticated reasoning ability in LLMs by using prompts that request step-by-step explanations on intermediate reasoning steps. This approach significantly enhances performance on complex reasoning tasks, particularly in **mathematical ?** **mathematic**, common-sense, and symbolic reasoning.

Building on this foundation, subsequent studies by Hsieh et al. (2023) and Jazbec et al. (2024) demonstrated that integrating reasoning chains directly into training datasets can significantly reduce the volume of data required during the distillation process. Hsieh refers to this process as *chain-of-thought distillation* [12, 14].

In this process, explanations are extracted from a teacher model regarding its decisions and then these rationales are included into the student model’s training data. The result is that student models can achieve performance comparable to traditional methods while requiring significantly fewer examples than conventional datasets lacking these explanations.

For our research, we apply this methodology specifically to translation tasks. In the context of machine translation, chain-of-thought distillation leverages large language models to generate synthetic explanations that clarify the relationship between source and target text in the training corpus. This approach is particularly promising for low-resource languages such as Javanese, where training data is inherently limited, so downstream model performance may be enriched through chain-of-thought distillation techniques.

Chapter 3

Methods

3.1 Goal

The goal of this study was to investigate the effect of different fine-tuning configurations on language models’ ability to translate Javanese to Indonesian, comparing the techniques of supervised fine-tuning on a small dataset, model distillation, and chain-of-thought distillation. To establish a good baseline for this translation task, we first tested various models (both Seq2Seq and generative models) on Javanese to Indonesian translation in a zero-shot setting, without any task-specific training.

For models with large enough context windows, we also tested their *many-shot* performance by adding the entire train examples into the context, checking if this helped improve translation quality.

After setting up these baselines, we moved to a step-by-step fine-tuning process, testing selected models on the same test set throughout. Our approach followed these stages:

1. Supervised fine-tuning using a dataset of 500 parallel examples.
2. Expanding the dataset through model distillation, where we used a top-performing zero-shot model to create additional training data. We explored two distinct

approaches:

- Adding 500 synthetic examples to the original dataset.
 - Adding approximately 1500 synthetic examples to the original dataset.
3. Creating a “chain-of-thought” dataset by asking a large language model to explain each translation in the original dataset, thereby providing models with enhanced contextual information during training.

At each stage, we picked a few of the best-performing models to continue to the next stage. Our objective was to create a fine-tuned small language model that worked as well as or better than larger models in zero-shot settings, while using fewer computational resources.

3.2 Dataset

Throughout this study, we utilized the *NusaX* dataset, which is a parallel corpus of example text excerpts across ten Indonesian languages [31]. For this study, we focused only on the Javanese and Indonesian examples, and left the rest of the languages for future work.

The *NusaX* dataset was structured with standard train, validation, and test splits, which we used as follows:

- The test split served as the benchmark for evaluating the quality of each model.
- The validation split was used to evaluate during the training process to prevent overfitting.
- The train split was the foundation for the various configurations of fine-tuning datasets that we constructed. In the first stage, we performed SFT on this data directly, and then we augmented it with synthetic examples and chain-of-thought reasoning steps in subsequent stages.

3.3 Models

Table 3.1 shows the list of models tested. For the zero-shot baseline testing, we used the largest set of models, spanning closed-source and large models that could not be run on single device. In the fine-tuning stages, we used models that were open-weight and runnable on a typical high-performance GPU with sufficient memory, such as the Nvidia A100 with 80 GB of VRAM.

Model Name	Parameters	Open	Single-GPU	Local
Google T5-base	223M	Y	Y	Y
Google mT5-base	580M	Y	Y	Y
Meta LLaMA 3.2 1B	1B	Y	Y	Y
Google Gemma 3 4B	4B	Y	Y	Y
Meta LLaMA 3.1 8B	8B	Y	Y	Y
Cohere4AI Aya 8B	8B	Y	Y	Y
Google Gemma 3 9B	9B	Y	Y	Y
Cohere4AI Aya 32B	32B	Y	Y	N
Meta LLaMA 3.3 70B	70B	Y	N	N
Meta LLaMA 405B	405B	Y	N	N
OpenAI GPT-4o-mini	*	N	N	N
OpenAI GPT-4o	*	N	N	N
Google Gemini 2.0 Flash	*	N	N	N
OpenAI o1-mini	*	N	N	N

Table 3.1: Model Specifications. *Open* means that the model’s weights are available online publicly, and its licensing permits further training. *Single-GPU* indicates the model can be trained on a single GPU with sufficient memory. *Local* signifies that the model is small enough to be run locally on a laptop computer.

3.3.1 Closed-Source Models

We included multiple closed-source models in our study, especially when establishing a baseline. These were often cutting-edge when first released and tended to be quite large. They were typically accessible through APIs, but their weights weren’t publicly available. This prevented local training and fine-tuning, though many did offer their own fine-tuning services.

3.3.2 Open-Weight Models

We evaluated various open-weight models across different parameter scales. Since our research involved fine-tuning, we needed direct access to model weights to customize hyperparameters and training configurations for the models we were adapting for the task of translation.

In some cases, newer small models may perform better than much larger models from just a year ago. This challenges the idea that bigger models are always better and also leaves room for tuned small models to outperform large models on the tests performed in this study.

For example, in the case of Google’s models, Gemma 3 4B was competitive with Gemma 2 27B, and Gemma 3 27B was competitive with Gemini 1.5 Pro, and Gemma 3 27B exceeded models such as LLaMA 3.1 405B in some benchmarks (e.g. *Chatbot Arena*¹), as of March 8, 2025 [27].

3.4 Dataset Construction

Our fine-tuning for Javanese-to-Indonesian translation involved distinct dataset configurations across three stages. This section describes how we prepared and structured the training data for each stage: supervised fine-tuning with the initial dataset of parallel text, model distillation with synthetic examples, and chain-of-thought distillation incorporating reasoning steps. We selected a few high performing small language models across a range of sizes to test these techniques.

3.4.1 Stage 1: Supervised Fine-Tuning on Initial Dataset

You said this above.

We began with the *NusaX* training split, which contained 500 parallel examples across ten languages [31]. From this dataset, we extracted the Javanese and Indonesian

¹<https://lmarena.ai/>

columns. The data was structured differently depending on the model type:

- **Sequence-to-sequence (Seq2Seq) models:** The dataset is structured with source and target text pairs, where “text” is the Javanese excerpt and “target” is the Indonesian excerpt.

```
{  
  "text": "Ibu uwis tahu kerja ing grab indonesia" (Javanese),  
  "target": "Ibu pernah bekerja di grab indonesia" (Indonesian)  
}
```

- **Generative models tuned for chat formatting:** The dataset incorporated a short instruction prompt.

```
{  
  "text": "Translate from Javanese to Indonesian:  
          Ibu uwis tahu kerja ing grab indonesia" (Javanese),  
  "target": "Ibu pernah bekerja di grab indonesia" (Indonesian)  
}
```

In this format, “text” is tokenized as the user’s input message to the chat model, and “target” is tokenized as the model’s expected response.

For all cases, we fine-tune the given model on all 500 of the Javanese-Indonesian examples in the *NusaX* parallel corpus.

3.4.2 Stage 2: Model Distillation

For the model distillation stage, we maintained the same formatting as Stage 1 while augmenting the *NusaX* dataset with synthetic data generated by a large language

model. We selected Google Gemini 2.0 Flash for this task due to its strong zero-shot performance and favorable licensing terms that permitted training derivative models on its outputs, unlike OpenAI’s GPT series. By incorporating synthetic data from this high-performing model, we aimed to distill its capabilities into smaller fine-tuned models.

To generate the synthetic examples, we provided the following prompt to Gemini 2.0 Flash:

```
You will help create a more comprehensive training set for a model
that translates Javanese to Indonesian.
```

```
Below is a list of Javanese text samples and their Indonesian
translations.
```

```
{examples}
```

```
Please generate 50 more rows of Javanese text samples and their
Indonesian translations, different from the examples above.
```

```
Each example should range from 1 to 3 sentences.
```

```
The examples should be in a variety of the formality levels, topics,
and grammatical structures (not just simple sentence constructions)
present in Javanese,
```

```
across the formality levels Ngoko, Ngoko Alus, Krama, and Krama Alus.
```

```
Return the examples in a JSON list of objects with two columns:
```

```
"javanese" and "indonesian".
```

```
Generate 50 examples.
```

In this prompt, the placeholder ‘*examples*’ initially contained the 500 Javanese-Indonesian pairs from the *NusaX* training split used in Stage 1. For subsequent generation requests, ‘*examples*’ was updated to include all previously generated pairs. This iterative approach ensured that newly generated examples were distinct from the existing ones.

We generated synthetic data in batches of 50 examples at a time to avoid exceeding the output token limit of Gemini 2.0 Flash (8192 tokens), even though it supported a large input context window (1,048,576 tokens).² This iterative generation process resulted in two distinct augmented datasets used for fine-tuning:

- **Augmented Dataset 1:** Combines the 500 original *NusaX* examples with 500 synthetic examples generated by Gemini 2.0 Flash, for a total of **1000 parallel pairs**.
- **Augmented Dataset 2:** Combines the 500 original *NusaX* examples with 994 synthetic examples generated by Gemini 2.0 Flash, for a total of **1494 parallel pairs**.

3.4.3 Stage 3: Chain-of-Thought Distillation

To perform chain-of-thought (CoT) distillation, we produced another augmented dataset. Instead of generating new synthetic parallel examples as in Stage 2, we used Gemini 2.0 Flash to generate a step-by-step reasoning chain (explanation) for each of the original 500 Javanese-Indonesian pairs from the *NusaX* training split. From this process, we created new data entries where each entry consisted of the source text, the target translation, and a detailed explanation outlining the translation steps. These new data entries were concatenated onto the initial 500 examples from the *NusaX* training split.

To generate these explanations, we prompted Gemini 2.0 Flash as follows:

```
You will help create a more comprehensive training set for a  
model that translates Javanese to Indonesian,  
incorporating chain-of-thought reasoning.
```

```
Below is a list of Javanese text samples and their
```

²<https://ai.google.dev/gemini-api/docs/models#gemini-2.0-flash>

Indonesian translations.

<examples> The <...> were not present in the previous prompt. Should they be?

{examples}

</examples>

For each example, prompt the model to generate a chain-of-thought for the translation, explaining the steps it took to arrive at the answer,

with phrase-level explanations.

Return the explanation in a JSON list of objects with three columns: "javanese", "indonesian", and "explanation", where "javanese" is the original Javanese text, "indonesian" is the translated Indonesian text, and "explanation" is the chain-of-thought explanation.

Do this for all of the examples provided.

We processed the original 500 Javanese-Indonesian pairs in batches of 25 to accommodate the output token limit of Gemini 2.0 Flash when generating the explanations.

Once the explanations were generated, we created new dataset entries for CoT distillation. Each entry followed this format, prompting the model to explain the translation process:

```
{
  "text": "Here is a Javanese sentence: Ibu uwis tahu kerja ing grab
          indonesia and its Indonesian translation: Ibu pernah
          bekerja di grab indonesia. Now, explain how it was
          translated.",
  "target": "\"Ibu\" translates directly to \"Ibu\" (Mother).
            \"uwis tahu\" translates to \"pernah\" (once).
```

```

    \"kerja\" translates to \"bekerja\" (work).
    \"ing grab indonesia\" translates to \"di Grab Indonesia\"
    (at Grab Indonesia).\"
}

```

We generated one such explanation entry for each of the original 500 pairs. These 500 explanation-focused entries were then combined with the original 500 *NusaX* pairs from Stage 1, resulting in a final CoT distillation dataset containing **1000 entries**.

In total, we created four datasets, summarized in Table 3.2. We proceeded to fine-tune open-weight models on these datasets and evaluated their performance.

Table 3.2: Summary of Constructed Datasets

Dataset	Technique	Size	Notes
Base Dataset	Supervised Fine-Tuning	500	Original parallel pairs from NusaX training split
Augmented Dataset 1	Model Distillation	1,000	500 original+500 synthetic pairs
Augmented Dataset 2	Model Distillation	1,494	500 original+994 synthetic pairs
CoT Dataset	Chain-of-Thought Distillation	1,000	500 original pairs+500 explanation entries

3.5 Fine-Tuning

This section outlines our fine-tuning approach for Javanese-to-Indonesian translation. We began with traditional sequence-to-sequence (Seq2Seq) models and then fine-tuned recent open-weight generative models. To balance computational efficiency with performance, we used Low-Rank Adaptation (LoRA) for parameter-efficient fine-tuning, enabling adaptation of the language models for translation without the computational demands of full-parameter fine-tuning.

The two sequence-to-sequence transformer-based models we fine-tuned were Google’s

T5 and multilingual T5 (mT5) models, at the *base* size of 220 million parameters, and 580 million parameters, respectively.

The generative models we fine-tuned were Gemma 3 4B, Cohere4AI Aya 8B, and Cohere4AI Aya 32B. Each of these had been pre-trained on numerous languages, but Javanese was not explicitly listed in this list of languages. That said, strong support for Indonesian was explicitly mentioned, as each of these models were post-trained with the focus of functioning in Indonesian. The Aya Expanse models were post-trained to support 23 languages, and Gemma 3 was post-trained to support 35 languages.

Since Javanese has a significant number of speakers (84 million [31]), it was possible that Javanese data was included in the pre-training for these models, but we did not know for certain.

3.5.1 Memory Considerations

Quantization

To fine-tune larger models within the constraints of a single GPU’s memory, we used quantization to reduce the memory occupied by the model. Quantization reduced the precision of model weights from their original 32-bit floating-point (FP32) representation to **lower bit formats**, significantly decreasing memory requirements while maintaining reasonable performance.

For most of our fine-tuned models, we utilized BF16 (Brain Floating Point) precision. BF16 maintains the same exponent size as FP32 but reduces the mantissa, allowing for adequate representation of large values while requiring half the memory. BF16 is a truncated version of FP32 that has been shown to achieve the same results as training with full FP32 precision [15].

The largest model we fine-tuned was Cohere4AI Aya 32B, which has a size too large to train on a single GPU. We applied 4-bit quantization (int4) to compress model

weights. The int4 quantization reduced the memory footprint by approximately 8x compared to full-precision training, enabling us to fine-tune the 32B parameter model on a single Nvidia A100 GPU with 80GB of VRAM.

Attention Mechanisms

Additionally, we utilized Flash Attention 2 [8] for fine-tuning both Aya models (8B and 32B). Flash Attention 2 modifies the standard transformer attention mechanisms by exploiting GPU memory hierarchy and improving work partitioning across thread blocks. This implementation reduces the quadratic memory requirements of standard attention to linear scaling, eliminates redundant memory operations, and achieves up to a 2x speedup over the original Flash Attention [8]. This enabled faster training and inference with longer sequences on limited hardware resources.

For the Gemma 3 4B model, we used “eager” attention as recommended by Google’s implementation guidelines.³ This recommendation originated from Gemma models utilizing a technique called “soft-capping” on attention and final logits to stabilize training [27]. To ensure soft-capping applied, Google recommended using eager attention for training runs on Gemma, which computed the full attention matrix at once and may have consumed more memory than optimized attention variations like Flash Attention 2.⁴

While quantization and specialized attention mechanisms can slightly reduce model performance, they enabled effective fine-tuning while making it possible to train these models quickly on limited hardware, such as the single Nvidia A100 GPU used in this study.

³https://ai.google.dev/gemma/docs/core/huggingface_text_finetune_qlora

⁴<https://huggingface.co/blog/gemma2#soft-capping-and-attention-implementations>

3.5.2 Parameter-Efficient Fine-tuning

To efficiently fine-tune the large generative models, we used Low-Rank Adaptation (LoRA) [13]. Full fine-tuning updates every weight in a model but is computationally expensive for large models.

LoRA offered a more efficient approach. It kept the original pre-trained model weights frozen and used products of low-rank matrix pairs in specific layers [13]. Instead of training all the original weights, LoRA only trained this set of low-rank matrices. The core idea was that the changes needed to adapt the model for most new tasks could be effectively captured by these low-rank updates, instead of requiring updates to all the parameters in a model.

This method significantly reduces the number of parameters that needed training and reduced the GPU memory needed for training. Hu et al. found that for GPT-3, this reduced parameters by a factor of 10,000 and GPU memory by a factor of 3 [13]. Despite training far fewer parameters, LoRA often achieved performance similar to or even better than full fine-tuning. A key advantage was that these small trained matrices could be merged back into the original weights after training, meaning LoRA added no latency when the model was used for inference. In this study, this enabled us to adapt large models to the task of Javanese-to-Indonesian translation effectively while using less computational resources and storage.

3.5.3 Training Decisions

Our training methodology involved fine-tuning each model across multiple epochs (5-10) and comparing the training and validation losses, using the given training dataset and the *NusaX* validation split, respectively. The loss curves often diverged, indicating potential overfitting to the training data. For each model, we selected the checkpoints with minimum validation loss from the training run to use as the basis for evaluations. See Figure 3.1 for an example loss plot comparing validation and

training loss.

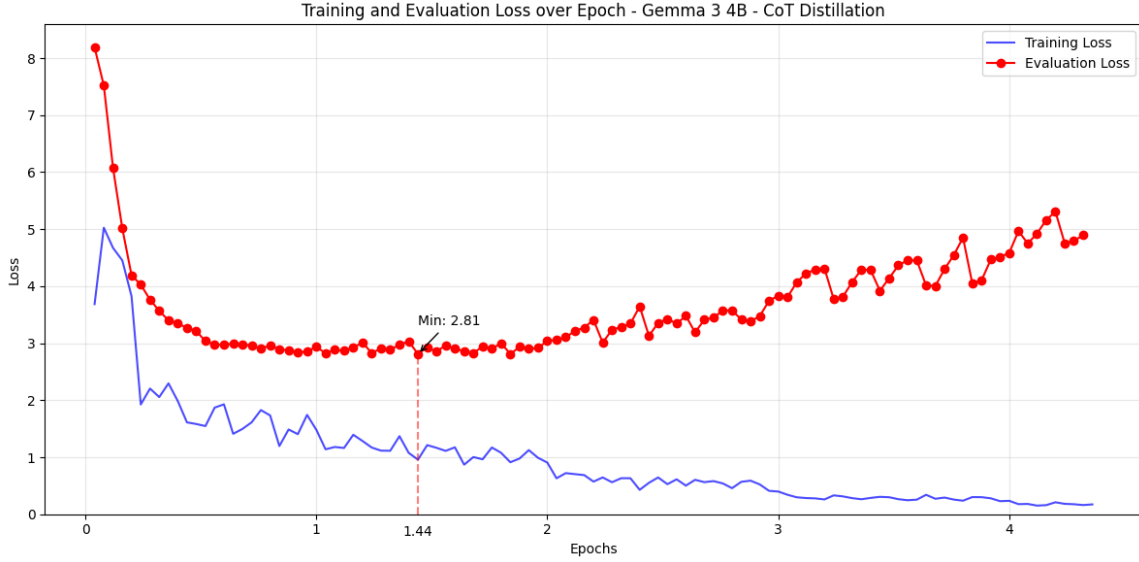


Figure 3.1: Training and validation loss curves for Gemma 3 4B model with Chain-of-Thought distillation. The checkpoint with the lowest validation loss was selected for evaluation.

For final performance evaluation, we ran inference on these selected checkpoints on each example in the held-out test set from *NusaX* and measured their relative performance (see Chapter 4)

The Appendix contains details on the specific hyperparameters used for each model, including learning rates, batch sizes, and other configuration choices that shaped our fine-tuning process.

Chapter 4

Evaluation

In this section, we evaluate the quality of the Javanese-to-Indonesian translations produced by the various models we are studying. We compare the performance of closed-weight and open-weight models in zero-shot and many-shot settings. We also evaluate models that were fine-tuned using the techniques described previously—supervised fine-tuning, model distillation, and chain-of-thought distillation—then compare their relative performance.

4.1 Evaluation Techniques

In this paper, we use four automatic evaluation techniques which capture different aspects of translation quality: BLEU [18], Translation Edit Rate (TER) [23], character n -gram F-score (chrF) [19], and BERTScore [32]. BLEU measures word-level n -gram precision with a brevity penalty, TER computes the minimum edits required to match a reference translation, and chrF calculates n -gram similarity at the character level. BERTScore uses contextual embeddings from pre-trained language models to compute similarity between candidate and reference sentences, providing an evaluation that is more semantically aware than surface-level n -gram approaches.

4.1.1 BLEU

Papineni et al. introduced BLEU (Bilingual Evaluation Understudy) in 2002 as a method to automatically evaluate machine translation quality instead of using human evaluation [18]. A BLEU score is computed by comparing candidate translations with one or more reference translations, measuring the precision of n-gram matches between them. It calculates the geometric mean of n-gram precisions (typically for n=1 to 4), with a brevity penalty applied to penalize overly short translations.

The BLEU score is calculated using the following equation:

Let r be the length of the reference corpus, and c be the length of the candidate translation. The brevity penalty BP is computed as follows:

$$BP = \begin{cases} 1 & \text{if } c > r \\ e^{(1-r/c)} & \text{if } c \leq r \end{cases} \quad (4.1)$$

The BLEU score is then computed as:

$$BLEU = BP \cdot \exp \left(\sum_{n=1}^N w_n \log p_n \right) \quad (4.2)$$

where p_n is the precision for n-grams of size n , w_n is the weight for each n-gram precision (typically uniform weights), and N is the maximum n-gram size considered (usually 4).

BLEU scores range from 0 to 1, where higher scores indicate higher quality translations, in that they have greater correspondence to the reference samples. Because BLEU is computed based on n -gram similarity, there are significant constraints regarding the meaning of the evaluation. Specifically, BLEU does not account for the substitution of synonyms semantically equivalent phrases. Hence, a BLEU score may be high for a translation test set which may not necessarily be semantically accurate.

Nonetheless, BLEU remains one of the standard evaluation techniques for translation due to its language independence and ease of computation [20].

In this paper, we use SacreBLEU [20] to compute the corpus-level BLEU score for translations. **Cutesy. What's next? CordonBLEU?** **SacreBLEU** provides a standardized implementation of BLEU that addresses reproducibility issues by clearly specifying tokenization and other processing details, making evaluation results more consistent and comparable across research [20]

Scores

Table 4.1 presents the BLEU scores for all evaluated models. In zero-shot testing, proprietary models exhibited superior performance, with Gemini 2.0 Flash and GPT-4o both achieving scores of 0.50, followed by GPT-4o-mini (0.46) and LLaMA 3.1 405B (0.45).

Notably, Gemini 2.0 Flash achieved the highest overall score of 0.63 in the many-shot test, substantially outperforming all other evaluated model configurations.

Among the smaller models, Gemma 3 4B initially scored 0.31 in zero-shot testing, which is 0.03 lower than Aya 32B’s 0.34, but consistently outperformed Aya 32B across all fine-tuning stages. Gemma 3 4B maintained a score of 0.53 throughout most fine-tuning stages, with a slight decrease to 0.52 during SFT-aug-1.

It is also worth noting that Gemma 3 4B’s fine-tuned score of 0.53 not only surpasses all models in zero-shot testing, but it also exceeds the performance of every tested model configuration except Gemini 2.0 Flash in the many-shot scenario, which provides strong support for the effectiveness of our fine-tuning approach, especially in resource-constrained environments.

The performance of the small sequence-to-sequence models also drastically improved after the first stage of fine-tuning. After SFT-NusaX (Stage 1), mT5-base achieved a score of 0.33 and T5-base reached 0.22. With only 580M parameters, mT5 outperforms several much larger zero-shot models, such as Gemma 3 4B (0.31),

LLaMA 3.1 8B (0.28), and Aya 8b (0.22).

Model Name	Zero-shot	Many-shot	SFT-NusaX	SFT-aug-1	SFT-aug-2	SFT-CoT
Gemini 2.0 Flash	0.50	0.63	-	-	-	-
GPT-4o	0.50	-	-	-	-	-
GPT-4o-mini	0.46	-	-	-	-	-
LLaMA 3.1 405B	0.45	-	-	-	-	-
o1-mini	0.40	-	-	-	-	-
LLaMA 3.3 70B	0.40	-	-	-	-	-
Gemma 9B	0.36	-	-	-	-	-
Cohere4AI Aya 32B	0.34	-	0.46	0.47	0.45	0.49
Gemma 3 4B	0.31	-	0.53	0.52	0.53	0.53
LLaMA 3.1 8B	0.28	-	-	-	-	-
Cohere4AI Aya 8B	0.22	-	0.42	0.45	0.45	0.43
LLaMA 3.2 1B	0.10	-	-	-	-	-
T5-base	0.03	-	0.22	-	-	-
mT5-base	0.00	-	0.33	-	-	-

Table 4.1: **BLEU scores** - Model performance across different training configurations on a test set of 400 entries. Models are sorted by zero-shot performance.

4.1.2 TER

The Translation Edit Rate (TER) method for automatic MT evaluation is an alternative to BLEU, measuring the number of edits required to transform a candidate translation output into a reference translation [23]. TER calculates the minimum number of edits needed, where edits include insertions, deletions, substitutions of single words, and shifts of word sequences. The score is computed as:

$$\text{TER} = \frac{\text{number of edits}}{\text{average number of reference words}} \quad (4.3)$$

Unlike BLEU, TER is an error metric for which lower scores indicate higher quality translations. Additionally, TER can capture word reordering through its “shift” op-

eration, which is when a contiguous sequence of words is moved to a different position within the translated output. This shift operation counts as a single edit regardless of how many words are moved, making it particularly effective at addressing a limitation of n-gram based metrics like BLEU that struggle with evaluating translations with different word orders but equivalent meanings.

Lots of language pairs must have such word-order issues, e.g., English vs German or Japanese.

Scores

Table 4.2 shows the TER scores for all models across different training configurations. Since TER is an error metric, lower scores indicate better performance. The results show a similar pattern to the BLEU scores, with larger models generally outperforming smaller ones in the zero-shot setting.

In the zero-shot configuration, Gemini 2.0 Flash and GPT-4o achieve the best performance with a TER score of 0.30, followed closely by GPT-4o-mini at 0.33. The performance gradually decreases with model size.

For Gemini 2.0 Flash, the many-shot approach improves the error rate from 0.30 to 0.23, which is the best score for any model configuration. Similar to BLEU, this indicates that providing examples significantly enhances translation quality.

The fine-tuned models again show significant improvements over their zero-shot counterparts. Gemma 3 4B improves from 0.51 to 0.30 after SFT, increased slightly to 0.31 during the first augmentation step, and then returned to 0.30 for the second augmentation and chain-of-thought distillation stages. Gemma 3 4B’s best score of 0.30 is the same as Gemini 2.0 Flash and Gpt-4o in the zero-shot testing setup.

Similarly, Cohere4AI Aya models show improvements, with Aya 32B improving from 0.47 to 0.38 after Stage 1, 0.39 and 0.38 for each of the augmentation condigu-rations, but scored 0.41 for the chain-of-thought distillation phase. Aya 8B improved from 0.60 to 0.43 after Stage 1, then decreased to 0.39 and 0.38 during the augmented training stages, but scored 0.41 during the chain-of-thought distillation phase.

After Stage 1 (SFT-NusaX), mT5 again outperformed the same larger zero-shot model configurations: Gemma 3 4B (0.51), LLaMA 3.1 8B (0.58), and Aya 8B (0.60).

The augmentation strategies (SFT-aug-1 and SFT-aug-2) and chain-of-thought approach (SFT-CoT) provide additional modest improvements for some models. Notably, Cohere4AI Aya 32B achieves its best performance with the SFT-CoT approach (0.34), while Aya 8B performs best with the dataset example augmentation strategies (0.38).

Model Name	Zero-shot	Many-shot	SFT-NusaX	SFT-aug-1	SFT-aug-2	SFT-CoT
Gemini 2.0 Flash	0.30	0.23	-	-	-	-
GPT-4o	0.30	-	-	-	-	-
GPT-4o-mini	0.33	-	-	-	-	-
LLaMA 3.1 405B	0.35	-	-	-	-	-
o1-mini	0.39	-	-	-	-	-
LLaMA 3.3 70B	0.42	-	-	-	-	-
Gemma 9B	0.45	-	-	-	-	-
Cohere4AI Aya 32B	0.47	-	0.38	0.37	0.38	0.34
Gemma 3 4B	0.51	-	0.30	0.31	0.30	0.30
LLaMA 3.1 8B	0.58	-	-	-	-	-
Cohere4AI Aya 8B	0.60	-	0.43	0.39	0.38	0.41
LLaMA 3.2 1B	0.76	-	-	-	-	-
T5-base	0.98	-	0.62	-	-	-
mT5-base	1.00	-	0.48	-	-	-

Table 4.2: **TER scores** - Model performance across different training configurations on a test set of 400 entries. Lower scores indicate better performance. Models are sorted by zero-shot performance.

4.1.3 chrF

Popović (2015) introduced chrF, character n -gram F-score, which is an alternative method for automatic MT evaluation [19]. Instead of using word-level n -grams, this technique calculates n -gram similarity at the character level.

The character n -gram F-score (chrF) is formed from the character n -gram precision (chrP) and the character n -gram recall (chrR). Given a translation hypothesis h and a reference r , chrP is the percentage of n -grams in h that are represented in r , and chrR is the percentage of n -grams in r that are also present in h .

The final parameter for chrF is β , which is the weighting of the precision relative to recall in the formula for chrF, which follows:

$$\text{chrF}_\beta = (1 + \beta^2) \cdot \frac{\text{chrP} \cdot \text{chrR}}{\beta^2 \cdot \text{chrP} + \text{chrR}} \quad (4.4)$$

where chrP is the character n -gram precision, chrR is the character n -gram recall, and β is the parameter that assigns β times more importance to recall than to precision [19].

Scores

Table 4.3 shows chrF scores across our model lineup with different training approaches.

Similar trends persist in comparison to the BLEU and TER results. Gemini 2.0 Flash scored the highest in the many-shot (0.81) and zero-shot (0.76) scenarios. That said, while the fine-tuned models show improved performance compared to their base models, none of the fine-tuned models (Gemma 3 4B, Aya 8B, Aya 32B) matched or exceeded the scores of zero-shot scores of the large proprietary models: Gemini 2.0 Flash (0.76) and GPT-4o (0.75). Still, it is worth noting that the fine-tuned versions of Gemma 3 4B, which scored 0.74 across each stage of fine-tuning, again outperformed many models of a significantly larger size. This includes LLaMA 3.1 405B (0.71), which is 100 times Gemma 3 4B’s size.

Model Name	Zero-shot	Many-shot	SFT-NusaX	SFT-aug-1	SFT-aug-2	SFT-CoT
Gemini 2.0 Flash	0.76	0.81	-	-	-	-
GPT-4o	0.75	-	-	-	-	-
GPT-4o-mini	0.73	-	-	-	-	-
LLaMA 3.1 405B	0.71	-	-	-	-	-
o1-mini	0.69	-	-	-	-	-
LLaMA 3.3 70B	0.67	-	-	-	-	-
Gemma 9B	0.64	-	-	-	-	-
Cohere4AI Aya 32B	0.64	-	0.72	0.70	0.70	0.73
Gemma 3 4B	0.61	-	0.74	0.74	0.74	0.74
LLaMA 3.1 8B	0.58	-	-	-	-	-
Cohere4AI Aya 8B	0.54	-	0.67	0.70	0.69	0.68
LLaMA 3.2 1B	0.40	-	-	-	-	-
T5-base	0.22	-	0.45	-	-	-
mT5-base	0.05	-	0.59	-	-	-

Table 4.3: **chrF scores** - Model performance across different training configurations on a test set of 400 entries. Higher scores indicate better performance. Models are sorted by zero-shot performance.

4.1.4 BERTScore

The final automated MT evaluation technique we use is BERTScore, which calculates the semantic similarity between text samples. It leverages pre-trained contextual embeddings from BERT models [32], which convert words and phrases into numerical vectors that capture their meaning and context. These embeddings represent semantic relationships in a high-dimensional space where similar meanings appear closer together, allowing BERTScore to detect semantic equivalence beyond surface-level word matching.

Unlike traditional n-gram based metrics such as BLEU, BERTScore computes a similarity score for each token in the candidate translation with each token in the reference translation, and then computes the cosine similarity between the candidate and reference sentence vectors.

BERTScore consists of three primary components:

$$R_{\text{BERT}} = \frac{1}{|r|} \sum_{r_i \in r} \max_{c_j \in c} \mathbf{r}_i^T \mathbf{c}_j \quad (4.5)$$

$$P_{\text{BERT}} = \frac{1}{|c|} \sum_{c_j \in c} \max_{r_i \in r} \mathbf{c}_j^T \mathbf{r}_i \quad (4.6)$$

$$F_{\text{BERT}} = 2 \frac{P_{\text{BERT}} \cdot R_{\text{BERT}}}{P_{\text{BERT}} + R_{\text{BERT}}} \quad (4.7)$$

where $\mathbf{r}_i$ and $\mathbf{c}_j$ are the contextual embeddings of tokens in the reference and candidate translation, respectively. R_{BERT} represents recall, P_{BERT} represents precision, and F_{BERT} is the F1 score that combines both.

BERTScore offers advantages over traditional metrics by capturing semantic equivalence through contextual embeddings instead of surface forms. This can better handle word reordering and therefore may correlate better with human judgments, especially for morphologically complex languages which are not well evaluated by n -gram models [32].

A key component to computing the BERTScore is the embedding model used. In our tests, we use the *XLM-RoBERTa-base* model, which is a BERT-based embedding model developed by Meta that surpasses other BERT embedding model variants on multilingual tasks [6].

XLM-RoBERTa was trained on 100 languages, including both Indonesian and Javanese, so it is sufficient for evaluating the semantic quality of the Indonesian translations produced during these tests [6].

Scores

Table 4.4 presents the median BERTScore values for all evaluated models. BERTScore was computed using the XLM-RoBERTa-base model, which provides contextual em-

beddings compatible with both Indonesian and Javanese languages. The results show that Gemini 2.0 Flash in the many-shot setting achieved the highest median score of 0.979.

Among the fine-tuned models, Gemma 3 4B consistently performed well across all fine-tuning stages, with median scores ranging from 0.965 to 0.967, which approaches the performance of proprietary models like GPT-4o (0.969). These results indicate that our fine-tuning approach significantly improved the semantic quality of translations from smaller open-weight models.

The progression of scores across fine-tuning stages shows incremental improvements, with the Chain-of-Thought distillation (SFT-CoT) generally yielding the highest median scores for both Aya 32B (0.963) and Gemma 3 4B (0.967) compared to their zero-shot performance (0.948 and 0.940 respectively).

Model Name	Zero-shot	Many-shot	SFT-NusaX	SFT-aug-1	SFT-aug-2	SFT-CoT
Gemini 2.0 Flash	0.969	0.979	-	-	-	-
GPT-4o	0.969	-	-	-	-	-
GPT-4o-mini	0.965	-	-	-	-	-
LLaMA 3.1 405B	0.960	-	-	-	-	-
o1-mini	0.957	-	-	-	-	-
LLaMA 3.3 70B	0.955	-	-	-	-	-
Gemma 3 9B	0.948	-	-	-	-	-
Cohere4AI Aya 32B	0.948	-	0.955	0.958	0.953	0.963
Gemma 3 4B	0.940	-	0.965	0.966	0.967	0.967
LLaMA 3.1 8B	0.936	-	-	-	-	-
Cohere4AI Aya 8B	0.930	-	0.944	0.953	0.950	0.951
LLaMA 3.2 1B	0.888	-	-	-	-	-
T5-base	0.842	-	0.913	-	-	-
mT5-base	0.785	-	0.938	-	-	-

Table 4.4: **Median BERTScore values** - Model performance across different training configurations using XLM-RoBERTa-base embeddings. Higher scores indicate better semantic similarity to reference translations. Models are sorted by zero-shot performance.

Figure 4.1 displays a visualization of alternative features of the BERTScore distributions, and it includes the median, mean, minimum, and maximum BERTScores across each of the model’s test set results.

Most models have a maximum BERTScore of 1.0, meaning that they produced at least one exact match to the reference translation.

It is clear again that Gemini 2.0 Flash many-shot performed the best, as it has the highest median, mean, and minimum BERTScore out of the model configurations tested. The highest minimum score is significant as it indicates that for all test cases, Gemini 2.0 Flash many-shot maintained a higher quality floor than other models.

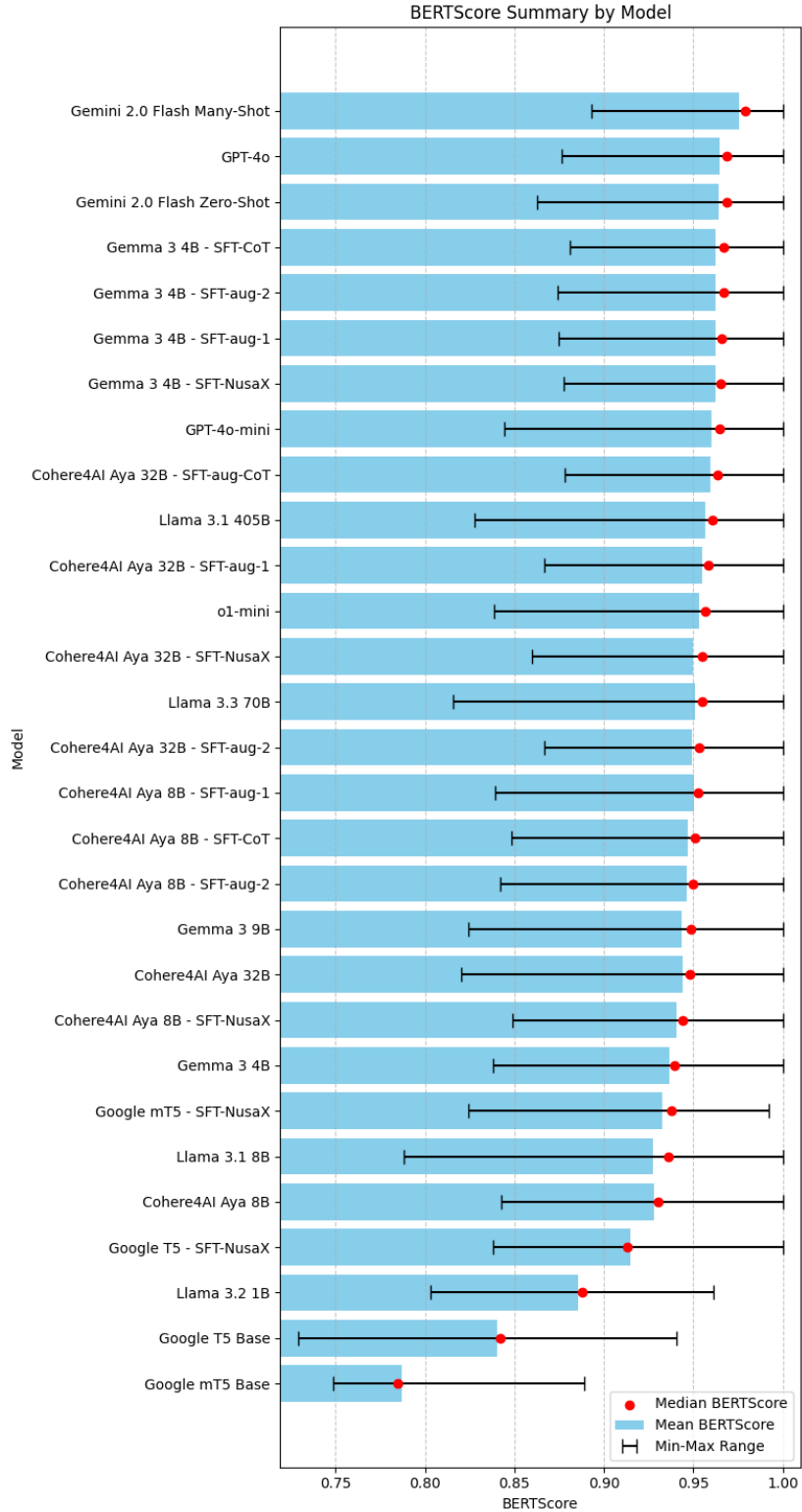


Figure 4.1: BERTScore results for different models using xlm-roberta-base embeddings. The figure shows that Gemini 2.0 Flash with many-shot learning achieved the highest average, median, and minimum scores, while most models reached a maximum score of 1.0, indicating perfect matches with reference translations in some cases.

Chapter 5

Discussion

5.1 Overall Performance

Across all four evaluation metrics (BLEU, TER, chrF, and BERTScore), the results consistently show that large proprietary models, particularly Gemini 2.0 Flash in a many-shot setting, achieve the highest translation quality, in terms of character- and word- based matching metrics, as well as semantic alignment metrics.

However, the results also show that fine-tuning experiments can produce significant performance gains for small language models on the task of Javanese translation. Supervised fine-tuning (SFT) dramatically boosted the performance of all fine-tuned models (Gemma 3 4B, Aya 8B, Aya 32B, mT5-base, T5-base) compared to their base models’ zero-shot performance. The most significant improvements were observed after the initial fine-tuning stage using the NusaX dataset (SFT-NusaX), and the performance on various metrics often further improved in the augmented dataset stages (SFT-aug-1, SFT-aug-2) or the chain-of-thought distillation stage (SFT-CoT), albeit only marginally.

A standout result is the performance of the fine-tuned Gemma 3 4B model. Despite being the smallest among the fine-tuned generative models, it consistently out-

performed the larger Aya models across all fine-tuning stages. Furthermore, the fine-tuned Gemma 3 4B often matched or exceeded the zero-shot performance of much larger models like GPT-4o and LLaMA 3.1 405B, particularly in BLEU and TER scores, only losing to Gemini 2.0 Flash. The metrics for which all fine-tuned Gemma 3 4B models lost to GPT-4o and Gemini 2.0 Flash (both zero-shot and many-shot) were chrF and BERTScore, which are arguably the most compelling metrics.

This emphasizes the potential of fine-tuning smaller, efficient models to achieve competitive performance in resource-constrained scenarios, compared to much larger models out-of-the-box.

5.2 Comparison of Techniques

While it is clear that SFT on a small dataset (NusaX train split of 500 parallel examples) was usually enough to significantly boost performance of the fine-tuned models on all metrics relative to their base models, it is less clear whether augmenting the SFT dataset with synthetic examples (model distillation) or adding reasoning chains to existing data (CoT distillation) is a more beneficial use of resources to further improve model performance on this task.

Model Distillation

Model distillation through synthetic data augmentation showed mixed effects across different models and metrics. For character- and word-based metrics (chrF, BLEU, TER), the benefits were inconsistent: Aya 8B generally showed progressive improvements (e.g., TER scores decreasing from 0.43 to 0.38 across distillation stages), while Gemma 3 4B maintained relatively stable performance (e.g., consistent BLEU scores around 0.52-0.53 and TER around 0.30-0.31), and Aya 32B occasionally experienced slight performance degradation (e.g., chrF declining from 0.72 to 0.70). These varied

responses suggest that the effectiveness of model distillation may depend on model architecture, with smaller models potentially benefiting more from additional synthetic examples. This aligns with a few claims from Zhu et al. (2024) which found that large models may overfit to imperfections in synthetic data and sacrifice performance [34].

In contrast, the BERTScore results revealed a more consistent positive impact of model distillation on semantic alignment across all models relative to initial SFT. Gemma 3 4B showed steady improvement from 0.940 after initial SFT to 0.966 for SFT-aug-2, Aya 8B improved from 0.930 to 0.953, and Aya 32B from 0.948 to 0.958. This consistent pattern may indicate that while surface-level features measured by BLEU, TER, and chrF may show variable responses to synthetic data augmentation, model distillation may be effective for enhancing semantic understanding and quality in translations.

However, the diminishing returns evident in subsequent distillation stages suggest the existence of a saturation threshold after which further synthetic data contributes only marginal improvements to translation quality.

Chain-of-Thought Distillation

Chain-of-Thought (CoT) distillation, similar to augmenting the dataset with more synthetic examples (model distillation), provided further modest improvements for some models and metrics on top of the initial SFT stage. The gains were often incremental compared to the large jump seen after the initial SFT on NusaX, suggesting diminishing returns for both data augmentation strategies.

CoT distillation appeared particularly beneficial for improving semantic similarity, consistently boosting BERTScore results, and leading to higher BERTScores than the example augmentation approach for Gemma 3 4B and Aya 32B.

5.2.1 Takeaways

Broadly, the methods added diminishing returns on top of the initial stage of supervised fine-tuning on NusaX.

Both model distillation and chain-of-thought (CoT) distillation extend supervised fine-tuning on NusaX, though with diminishing returns. They’re variants of distilling a *teacher* model’s knowledge: model distillation uses dataset examples to prompt similar outputs, expanding the data’s breadth, while CoT distillation extracts detailed reasoning traces from the initial examples, breaking them into step-by-step explanations. Model distillation is quantity-focused, scaling up the volume of training data with parallel examples. In contrast, CoT is quality-focused, enhancing the depth of each example by capturing the teacher’s internal logic—e.g., not just translating a sentence, but explaining why specific words or structures were chosen. Still, both remain tied to the original data, refining or reinterpreting it rather than introducing entirely new perspectives, so it is not surprising that the performance improvements are broadly marginal over the initial SFT stage of “original” data.

Saturation

The broad convergence of scores across fine-tuning stages for each model supports Zhu et al. (2024)’s findings on diminishing returns in machine translation fine-tuning. Their work showed that while LLaMA-2 achieved strong translation performance with just 32 examples, scaling to 75,000 samples yielded progressively smaller gains [34]. They tied this to Zhou et al. (2023)’s “superficial alignment hypothesis,” which argues that language capabilities are primarily acquired during pre-training, with fine-tuning merely **mentioned earlier. explain better.** aligning output formats rather than teaching new skills [33, 34].

Our results display this ceiling effect. The models may have reached a saturation point where they maximized their pre-trained representations, and additional fine-tuning data or techniques produced only marginal improvements. Significant per-

formance improvements would likely require extended pre-training with more diverse data before applying these fine-tuning approaches.

5.3 Common Issue: Formality Mismatch

During evaluation, we identified a significant issue related to formality levels in translation between Javanese and Indonesian, especially regarding pronoun usage. Our analysis revealed that the *NusaX* dataset, across all sections, contains primarily informal Javanese text (known as the *Ngoko* level in Javanese language hierarchy) rather than formal Javanese text (the *Krama* honorific level). However, the parallel examples in Indonesian are often in the formal honorific level instead of the informal one.

This characteristic leads to a potential mismatch during translation. For instance, the dataset often maps the informal Javanese “aku” (“I”) to the formal Indonesian “saya,” instead of preserving the informal level which might be represented by “aku” in informal Indonesian contexts. Similarly, the informal Javanese “dhewe” (“we”) is frequently mapped to the formal and exclusive Indonesian “kami,” rather than the informal and inclusive “kita.”

Table 5.1 shows an example of this pronoun formality mismatch. The Gemma 3 4B base model, when tested in a zero-shot setting, often produced translations that correctly maintained the informal register but were penalized according to the ground truth, which favored a shift to formal Indonesian. Conversely, models fine-tuned on the *NusaX* training split, such as the Gemma 3 4B CoT model in Table 5.1, learned to consistently map informal Javanese inputs to formal Indonesian outputs, aligning better with the dataset’s ground truth but potentially diverging from real-world usage where formality preservation might be preferred.

This suggests that fine-tuning specifically on the *NusaX* dataset, which contains

these inherent formality mismatches, caused models to learn an inappropriate translation pattern—converting informal Javanese to formal Indonesian. While this pattern also aligned with the NusaX test set and resulted in higher scores, it may not accurately preserve the intended formality level in real-world translation scenarios where maintaining the original register would be preferred.

Example

Consider the following translation example, taken from the test dataset and results:

Input (Javanese)	Informal Javanese	“ Aku nembe bae ganti kartu sing rusak, nanging tetep ora ana sinyal.”
Ground-Truth (Indonesian)	Formal Indonesian	“ Saya baru saja ganti kartu yang rusak, tapi tetap tidak ada sinyal.”
Gemma 3 4B Base Output	Informal Indonesian	“ Aku baru saja mengganti kartu yang rusak, tapi masih tidak ada sinyal.”
Gemma 3 4B CoT Output	Formal Indonesian	“ Saya baru saja ganti kartu yang rusak, tapi tetap tidak ada sinyal.”

Table 5.1: Example of pronoun formality differences in translation

In this case, the base model correctly preserves the informal pronoun “Aku,” while the CoT model aligns with the ground truth by translating it to the formal “Saya.” This highlights the tendency of the fine-tuned model to adopt the formality shift present in the training data.

5.4 Model Size Considerations

While Gemini 2.0 Flash and GPT-4o often exceeded the quality of fine-tuned smaller models, it is important to emphasize that using our techniques, we successfully fine-tuned Gemma 3 4B to nearly match their translation quality across character preci-

sion, error rate, and semantic similarity metrics.

A paper from Microsoft estimates GPT-4o to have approximately 200 billion parameters [1]. This means that our 4 billion parameter Gemma 3 4B model approaches the performance level of a model two orders of magnitude larger in size.

A 4B parameter model like Gemma 3 4B can be run on edge devices such as modern laptop and mobile phones, significantly increasing access to high-quality translation services in remote parts of Java where Javanese is predominantly spoken. Such a model, fine-tuned specifically for Javanese-to-Indonesian translation, can serve the vast population of Javanese speakers who wish to interface with the rest of Indonesia or engage with global media.

This process may also generalize to other low-resource languages, opening doors for many specialized small models to serve populations across the world who currently lack access to high-quality translation technologies.

Chapter 6

Conclusions and Future Work

This research aimed to enhance machine translation capabilities for low-resource Indonesian languages, specifically focusing on Javanese-to-Indonesian translation, by leveraging supervised fine-tuning (SFT) and knowledge distillation techniques. Our investigation compared the performance of various sequence-to-sequence and generative language models, evaluating the effectiveness of SFT, model distillation (synthetic data augmentation), and Chain-of-Thought (CoT) distillation against large proprietary models and zero-shot baselines.

The results consistently demonstrated that while large proprietary models like Gemini 2.0 Flash (using many-shot prompting) achieved the highest overall translation quality across metrics like BLEU, TER, chrF, and BERTScore, fine-tuning significantly boosted the performance of smaller, open-source models. Initial SFT using the NusaX dataset provided substantial gains over zero-shot performance for all tested models (Gemma 3 4B, Aya 8B, Aya 32B, mT5-base, T5-base).

Notably, the fine-tuned Gemma 3 4B model, despite being the smallest generative model evaluated, achieved performance comparable to, and occasionally surpassing, much larger models like GPT-4o and LLaMA 3.1 405B in their zero-shot capacity, particularly regarding BLEU and TER scores. This emphasizes the potential of fine-

tuning smaller models to deliver competitive translation quality.

Further enhancements through model distillation and CoT distillation yielded mixed results. While both techniques often led to incremental improvements, especially in semantic similarity (BERTScore), the gains were marginal compared to the initial SFT boost. This suggests diminishing returns and aligns with the “superficial alignment hypothesis,” indicating that performance might be capped by the knowledge acquired during pre-training, with fine-tuning primarily adapting the model to the task format. Model distillation showed inconsistent effects on surface metrics but consistently improved BERTScore, whereas CoT distillation also provided modest gains, particularly enhancing semantic alignment.

A key challenge identified was the formality mismatch in the NusaX dataset, where informal Javanese was often paired with formal Indonesian translations. Fine-tuned models learned this pattern, potentially leading to translations that, while scoring well against the test set, might not preserve the intended register in real-world applications.

6.1 Implications and Significance

The success in fine-tuning a 4-billion parameter model (Gemma 3 4B) to approach the translation quality of models orders of magnitude larger (like GPT-4o) has significant implications. It demonstrates the feasibility of developing high-performance, resource-efficient translation models for low-resource languages. Such models can be deployed on edge devices like laptops and mobile phones, increasing accessibility for speakers in regions with limited connectivity, such as parts of Java.

This work aims to help Javanese speakers better access online information and interact with content from diverse cultures. It may also contribute to improving access to modern technologies for low-resource language communities and support

language preservation initiatives.

6.2 Future Work

Several avenues for future research remain:

- **Scaling Data prior to Distillation:** Investigating whether combining distillation techniques (both model and CoT) with significantly larger datasets (beyond the initial NusaX split and augmentations) can overcome the observed performance plateaus and yield further improvements.
- **Extended Pre-training:** Exploring the impact of extending the pre-training phase of models like Gemma or Aya with substantial amounts of Javanese text before applying SFT and distillation. This could potentially raise the performance ceiling imposed by limited pre-training exposure.
- **Addressing Formality Mismatch:** Developing methods to mitigate the formality mismatch issue, perhaps by curating datasets with consistent formality levels or incorporating formality control mechanisms into the fine-tuning process.
- **Application to Other Languages:** Applying the successful fine-tuning methodologies identified here to other low-resource Indonesian languages present in datasets like NusaX, further validating the approach’s generalizability.

Addressing these areas could lead to even more robust and practical translation solutions for Javanese and other low-resource languages across the world.

Appendix A

Code

Bibliography

- [1] A. B. Abacha, W. wai Yim, Y. Fu, Z. Sun, M. Yetisgen, F. Xia, and T. Lin. Medec: A benchmark for medical error detection and correction in clinical notes, 2025.
- [2] D. Bahdanau, K. Cho, and Y. Bengio. Neural machine translation by jointly learning to align and translate, 2016.
- [3] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei. Language models are few-shot learners, 2020.
- [4] S. Cahyawijaya, H. Lovenia, A. F. Aji, G. I. Winata, B. Wilie, R. Mahendra, C. Wibisono, A. Romadhony, K. Vincentio, F. Koto, J. Santoso, D. Moeljadi, C. Wirawan, F. Hudi, I. H. Parmonangan, I. Alfina, M. S. Wicaksono, I. F. Putra, S. Rahmadani, Y. Oenang, A. A. Septiandri, J. Jaya, K. D. Dhole, A. A. Suryani, R. A. Putri, D. Su, K. Stevens, M. N. Nityasya, M. F. Adilazuarda, R. Ignatius, R. Diandaru, T. Yu, V. Ghifari, W. Dai, Y. Xu, D. Damapuspita, C. Tho, I. M. K. Karo, T. N. Fatyanosa, Z. Ji, P. Fung, G. Neubig, T. Baldwin,

- S. Ruder, H. Sujaini, S. Sakti, and A. Purwarianti. Nusacrowd: Open source initiative for indonesian nlp resources, 2023.
- [5] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio. Learning phrase representations using rnn encoder-decoder for statistical machine translation, 2014.
- [6] A. Conneau, K. Khandelwal, N. Goyal, V. Chaudhary, G. Wenzek, F. Guzmán, E. Grave, M. Ott, L. Zettlemoyer, and V. Stoyanov. Unsupervised cross-lingual representation learning at scale, 2020.
- [7] J. Dang, S. Singh, D. D’souza, A. Ahmadian, A. Salamanca, M. Smith, A. PEPpin, S. Hong, M. Govindassamy, T. Zhao, S. Kublik, M. Amer, V. Aryabumi, J. A. Campos, Y.-C. Tan, T. Kocmi, F. Strub, N. Grinsztajn, Y. Flet-Berliac, A. Locatelli, H. Lin, D. Talupuru, B. Venkitesh, D. Cairuz, B. Yang, T. Chung, W.-Y. Ko, S. S. Shi, A. Shukayev, S. Bae, A. Piktus, R. Castagné, F. Cruz-Salinas, E. Kim, L. Crawhall-Stein, A. Morisot, S. Roy, P. Blunsom, I. Zhang, A. Gomez, N. Frosst, M. Fadaee, B. Ermiş, A. Üstün, and S. Hooker. Aya expand: Combining research breakthroughs for a new multilingual frontier, 2024.
- [8] T. Dao. Flashattention-2: Faster attention with better parallelism and work partitioning, 2023.
- [9] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding, 2019.
- [10] G. Dong, H. Yuan, K. Lu, C. Li, M. Xue, D. Liu, W. Wang, Z. Yuan, C. Zhou, and J. Zhou. How abilities in large language models are affected by supervised fine-tuning data composition, 2024.
- [11] J. Howard and S. Ruder. Universal language model fine-tuning for text classification, 2018.

- [12] C.-Y. Hsieh, C.-L. Li, C.-K. Yeh, H. Nakhost, Y. Fujii, A. Ratner, R. Krishna, C.-Y. Lee, and T. Pfister. Distilling step-by-step! outperforming larger language models with less training data and smaller model sizes, 2023.
- [13] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. Lora: Low-rank adaptation of large language models, 2021.
- [14] M. Jazbec, M. Xia, A. Mallick, D. Madrigal, D. Han, S. Kessler, and V. Rühle. On efficient distillation from LLMs to SLMs. In *NeurIPS 2024 Workshop on Fine-Tuning in Modern Machine Learning: Principles and Scalability*, 2024.
- [15] D. Kalamkar, D. Mudigere, N. Mellempudi, D. Das, K. Banerjee, S. Avancha, D. T. Vooturi, N. Jammalamadaka, J. Huang, H. Yuen, J. Yang, J. Park, A. Heinecke, E. Georganas, S. Srinivasan, A. Kundu, M. Smelyanskiy, B. Kaul, and P. Dubey. A study of bfloat16 for deep learning training, 2019.
- [16] Y. Kim and A. M. Rush. Sequence-level knowledge distillation, 2016.
- [17] OpenAI. Gpt-4 technical report, 2024.
- [18] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu. Bleu: a method for automatic evaluation of machine translation. In *Proceedings of the 40th Annual Meeting on Association for Computational Linguistics, ACL '02*, page 311–318, USA, 2002. Association for Computational Linguistics.
- [19] M. Popović. CHRF: Character n-gram F-score for automatic MT evaluation. In *Proceedings of the Tenth Workshop on Statistical Machine Translation*, pages 392–395, Lisboa, Portugal, 17-18 September 2015. Association for Computational Linguistics.
- [20] M. Post. A call for clarity in reporting bleu scores, 2018.

- [21] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever. Improving language understanding by generative pre-training. *arXiv preprint arXiv:1806.04582*, 2018.
- [22] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever. Language models are unsupervised multitask learners. *OpenAI Technical Report*, 2019.
- [23] M. Snover, B. Dorr, R. Schwartz, L. Micciulla, and J. Makhoul. A study of translation edit rate with targeted human annotation. In *Proceedings of the 7th Conference of the Association for Machine Translation in the Americas*, pages 223–231, Cambridge, MA, August 2006. Association for Machine Translation in the Americas.
- [24] I. Sutskever, O. Vinyals, and Q. V. Le. Sequence to sequence learning with neural networks, 2014.
- [25] G. Tanzer, M. Suzgun, E. Visser, D. Jurafsky, and L. Melas-Kyriazi. A benchmark for learning to translate a new language from one grammar book, 2024.
- [26] G. G. Team. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context, 2024.
- [27] G. G. Team. Gemma 3 technical report, 2025.
- [28] N. Team, M. R. Costa-jussà, J. Cross, O. Çelebi, M. Elbayad, K. Heafield, K. Hefernan, E. Kalbassi, J. Lam, D. Licht, J. Maillard, A. Sun, S. Wang, G. Wenzek, A. Youngblood, B. Akula, L. Barrault, G. M. Gonzalez, P. Hansanti, J. Hoffman, S. Jarrett, K. R. Sadagopan, D. Rowe, S. Spruit, C. Tran, P. Andrews, N. F. Ayan, S. Bhosale, S. Edunov, A. Fan, C. Gao, V. Goswami, F. Guzmán, P. Koehn, A. Mourachko, C. Ropers, S. Saleem, H. Schwenk, and J. Wang. No language left behind: Scaling human-centered machine translation, 2022.

- [29] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need, 2017.
- [30] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou. Chain-of-thought prompting elicits reasoning in large language models, 2023.
- [31] G. I. Winata, A. F. Aji, S. Cahyawijaya, R. Mahendra, F. Koto, A. Romadhony, K. Kurniawan, D. Moeljadi, R. E. Prasajo, P. Fung, T. Baldwin, J. H. Lau, R. Sennrich, and S. Ruder. Nusax: Multilingual parallel sentiment dataset for 10 indonesian local languages, 2022.
- [32] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi. Bertscore: Evaluating text generation with bert, 2020.
- [33] C. Zhou, P. Liu, P. Xu, S. Iyer, J. Sun, Y. Mao, X. Ma, A. Efrat, P. Yu, L. Yu, S. Zhang, G. Ghosh, M. Lewis, L. Zettlemoyer, and O. Levy. Lima: Less is more for alignment. *arXiv preprint arXiv:2305.11206*, 2023.
- [34] D. Zhu, P. Chen, M. Zhang, B. Haddow, X. Shen, and D. Klakow. Fine-tuning large language models to translate: Will a touch of noisy data in misaligned languages suffice?, 2024.